Working with Databricks notebooks

DATA TRANSFORMATION WITH SPARK SQL IN DATABRICKS



Disha Mukherjee
Lead Data Engineer

datacamp

Your instructor

Disha Mukherjee

- Lead Data Engineer
- Worked at Ford Credit, Just Eat and PayU



COURSE

Introduction to Databricks SQL

Intermediate

Learn Databricks SQL for data engineering, analytics, and real-time data workflows in the lakehouse architecture.

COURSE

Introduction to PySpark

Intermediate

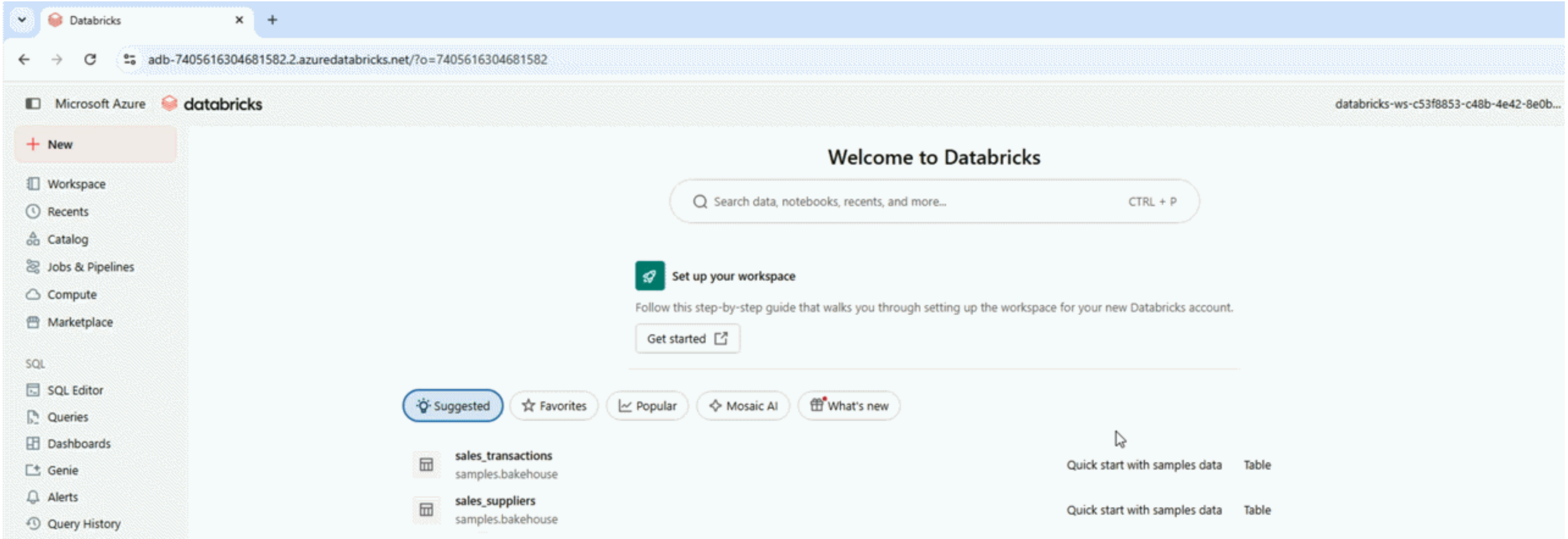
Master PySpark to handle big data with ease—learn to process, query, and optimize massive datasets for powerful analytics!

- Will apply skills inside Databricks

What to expect

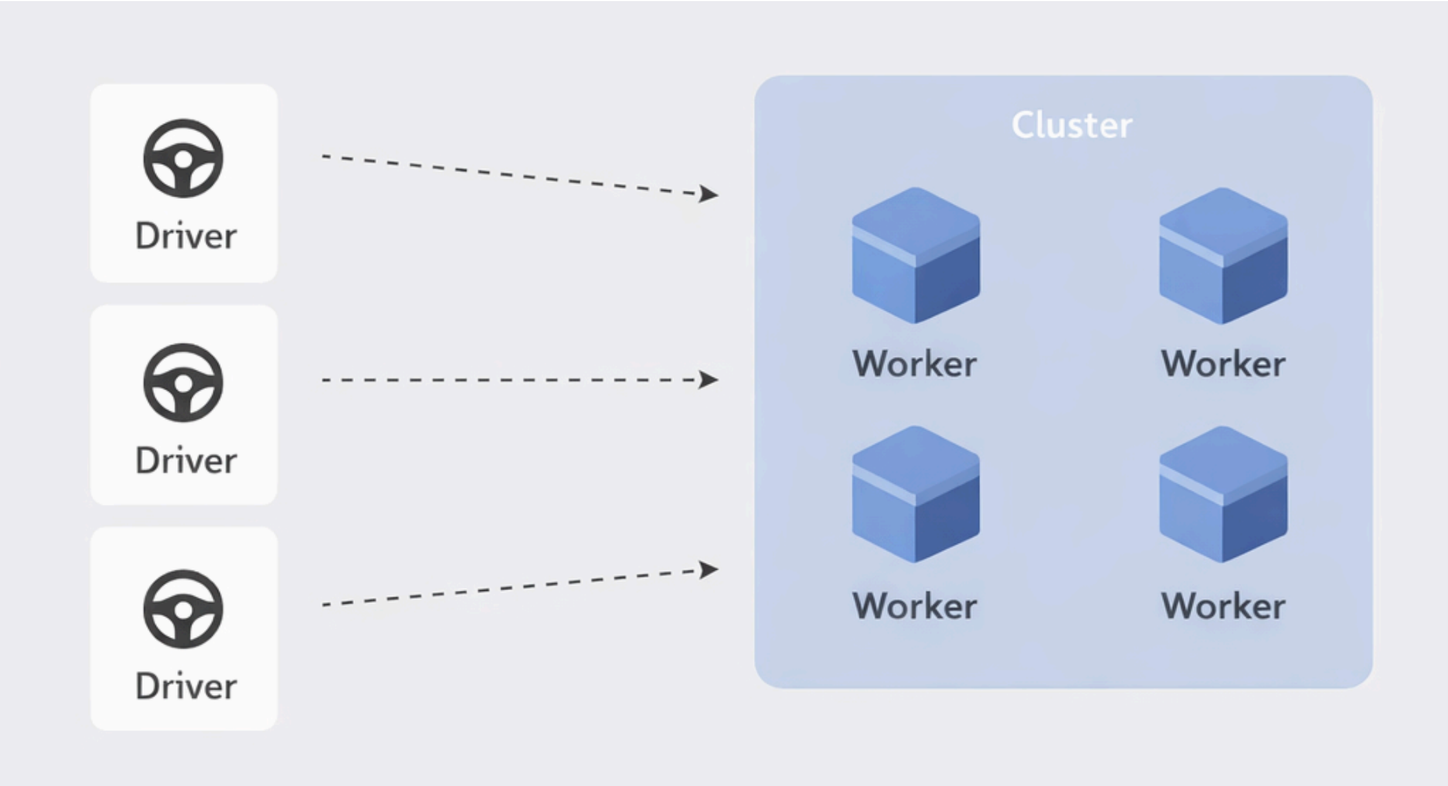


What is Databricks?



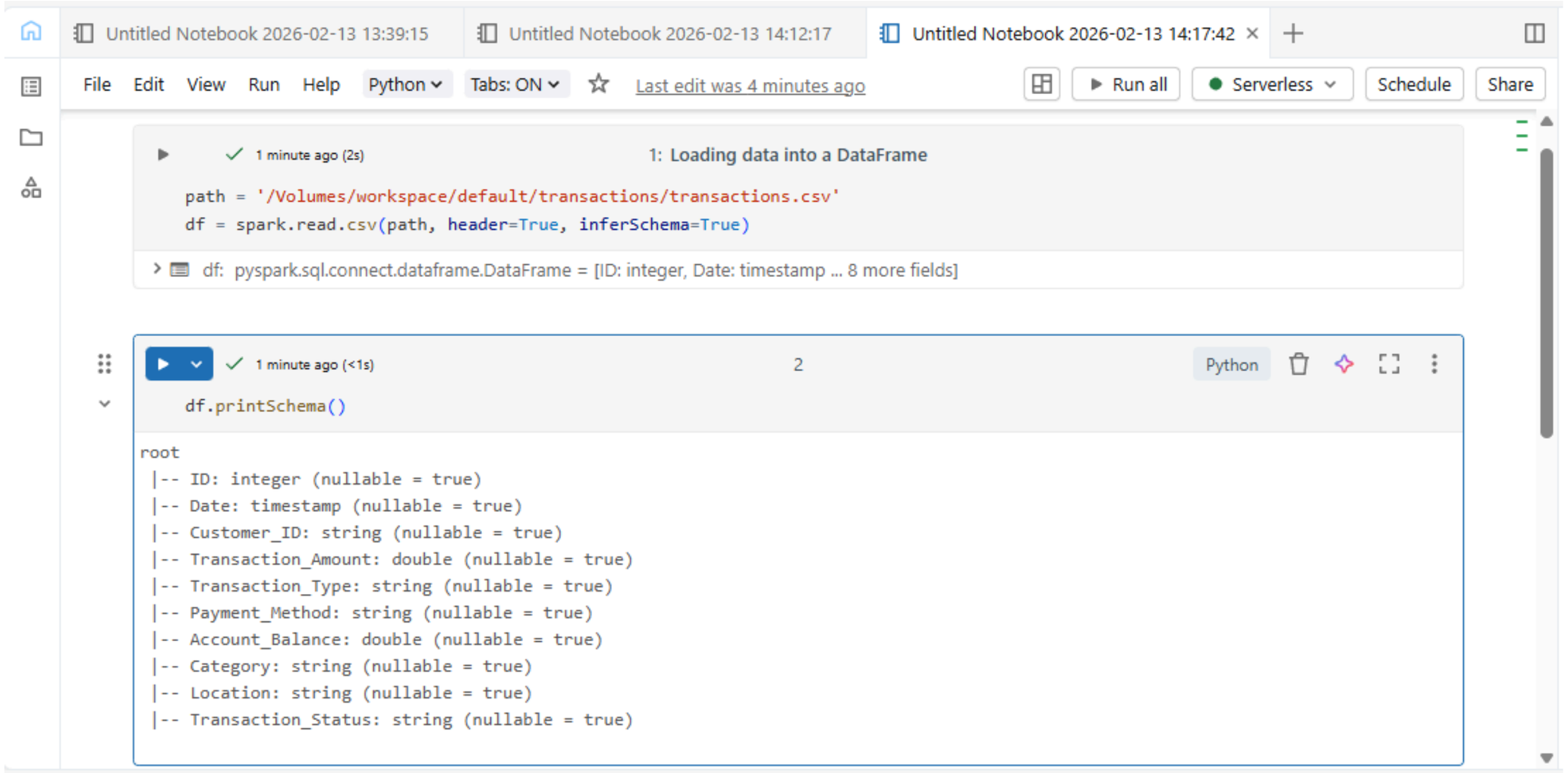
- Provides tools for engineering and analytics 📊
- Handles provisioning and scaling for us 🙌

Clusters and Spark

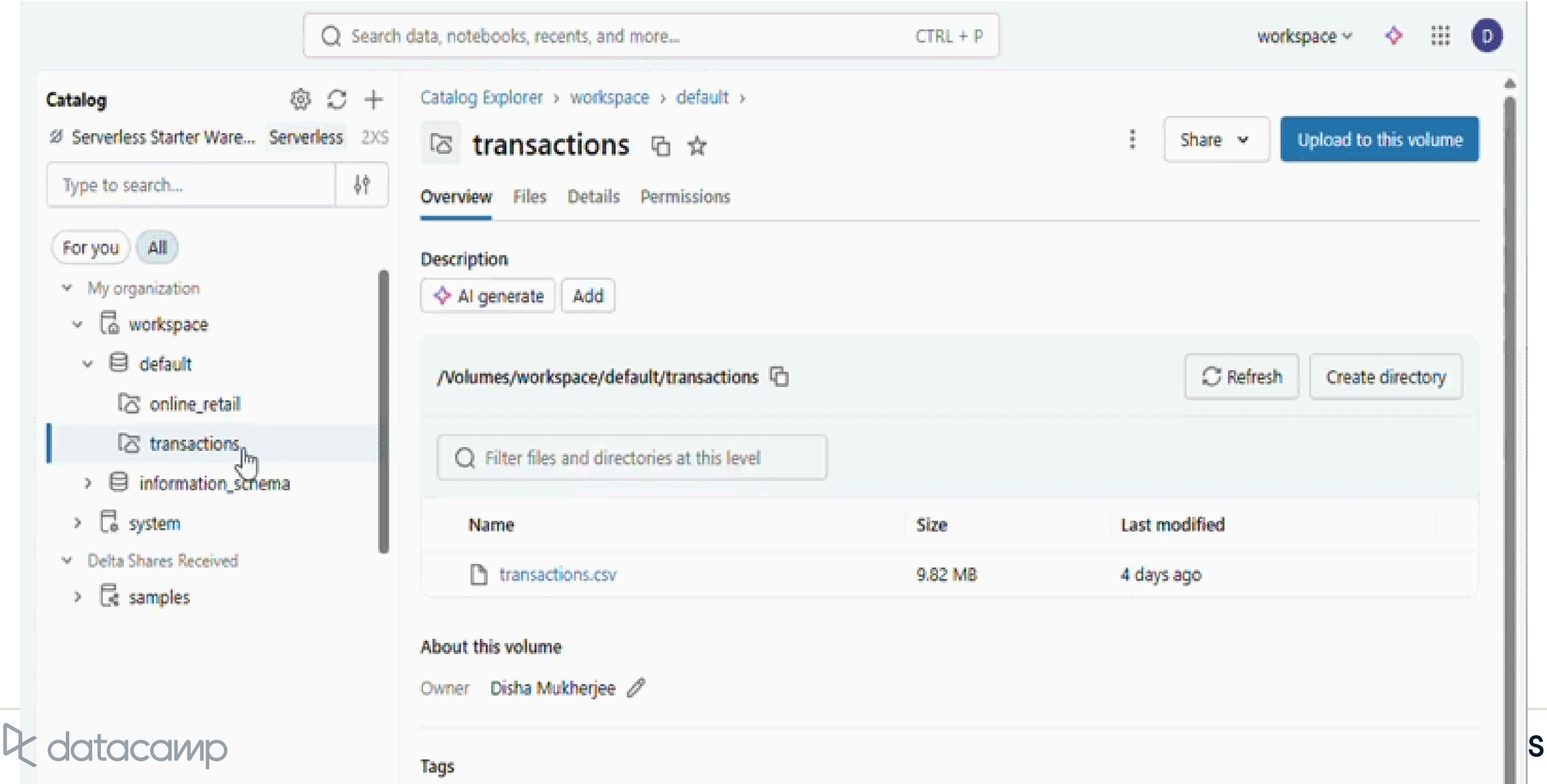


- Spark handles large datasets efficiently
- Exercises use **serverless compute** - same architecture, no setup

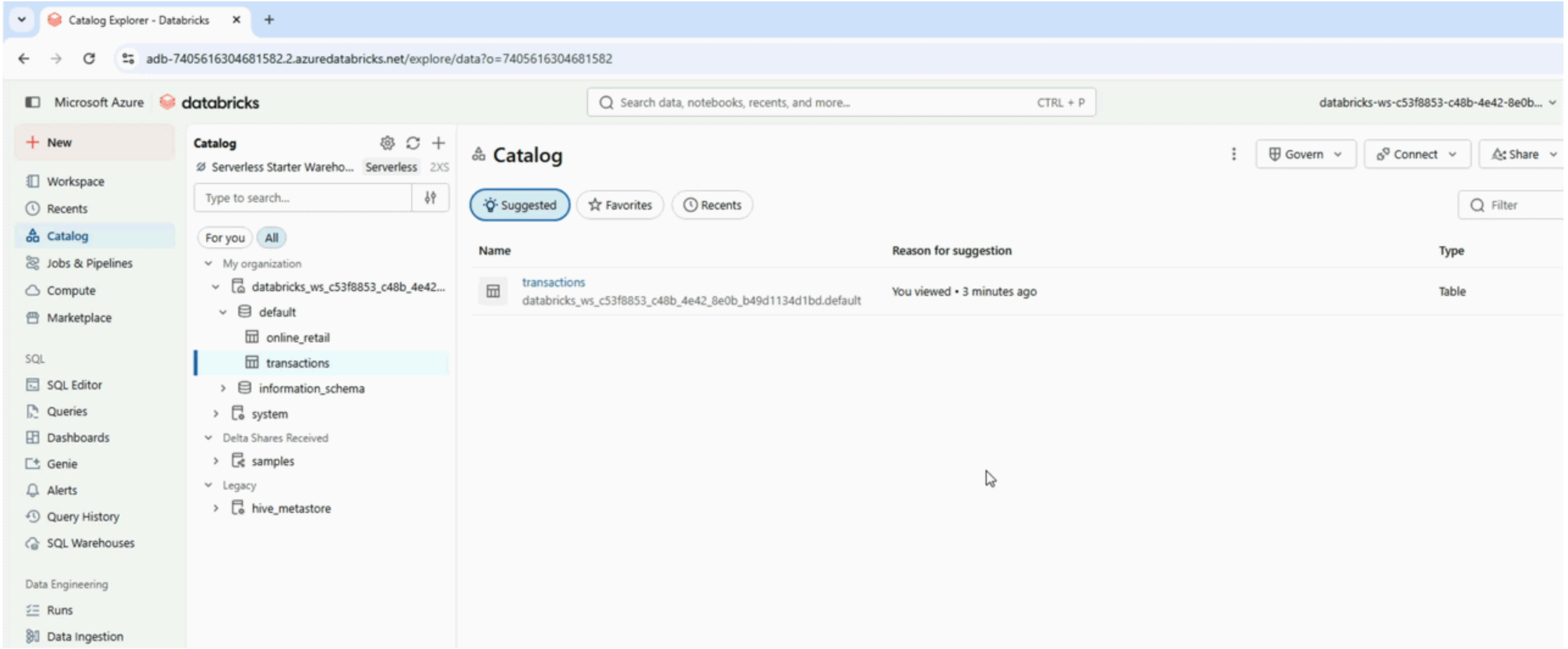
Databricks notebooks



Dataset introduction



Unity Catalog Volumes



Loading data into a DataFrame

```
path = "/Volumes/.../default/transactions.csv"
df = spark.read.csv(path, header=True, inferSchema=True)
```

```
df:pyspark.sql.connect.dataframe.DataFrame
    ID:integer
    Date:timestamp
    Customer_ID:string
    ....
    Transaction_Status:string
```

Inspecting the DataFrame

```
df.printSchema()
```

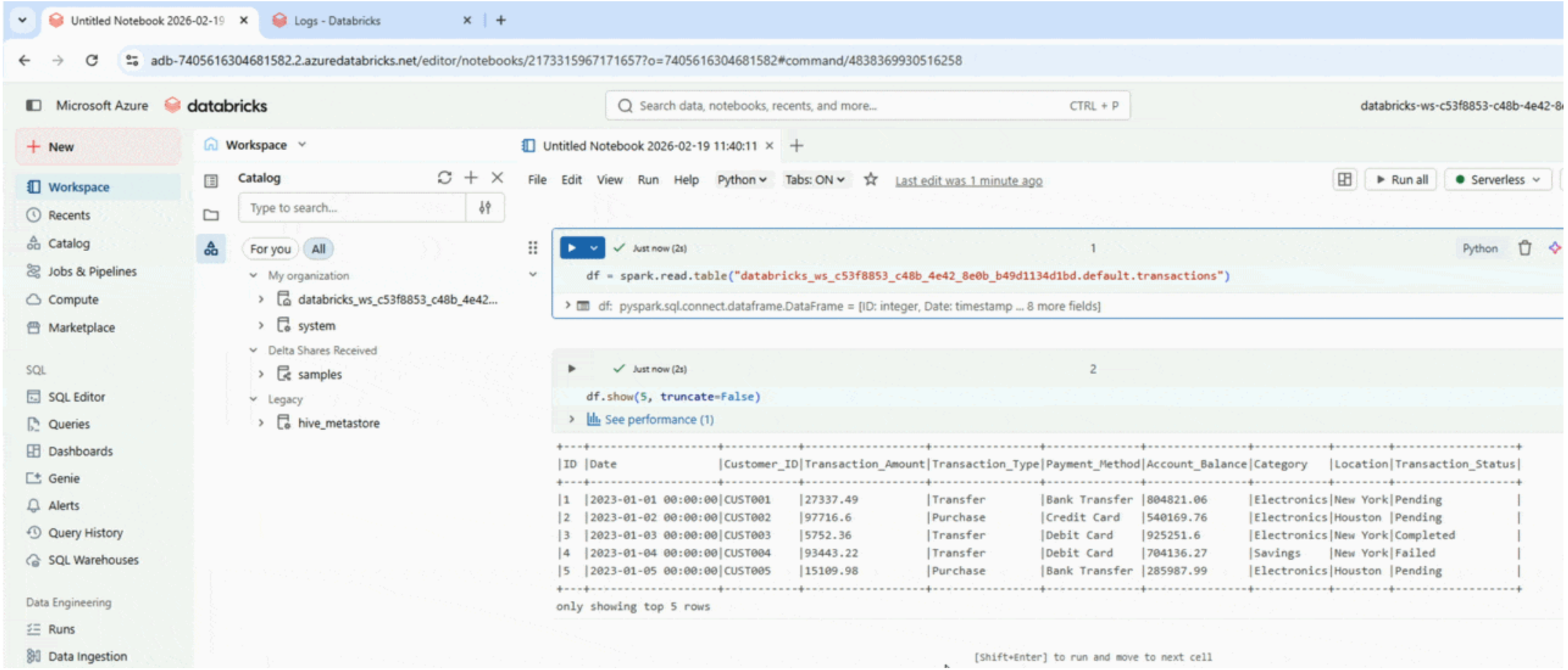
```
root
 |-- ID: integer (nullable = true)
 |-- Date: timestamp (nullable = true)
 ...
 |-- Transaction_Status: string (nullable = true)
```

Previewing the data

```
df.show(5, truncate=False)
```

```
+---+-------------------+----------+------------------+----------------+--------------+--------------+----------+--------+------------------+
|ID |Date               |Customer_ID|Transaction_Amount|Transaction_Type|Payment_Method|Account_Balance|Category  |Location|Transaction_Status|
+---+-------------------+----------+------------------+----------------+--------------+--------------+----------+--------+------------------+
|1  |2023-01-01 00:00:00|CUST001    |27337.49          |Transfer        |Bank Transfer |804821.06      |Electronics|New York|Pending           |
|2  |2023-01-02 00:00:00|CUST002    |97716.6           |Purchase        |Credit Card   |540169.76      |Electronics|Houston |Pending           |
|3  |2023-01-03 00:00:00|CUST003    |5752.36           |Transfer        |Debit Card    |925251.6       |Electronics|New York|Completed         |
|4  |2023-01-04 00:00:00|CUST004    |93443.22          |Transfer        |Debit Card    |704136.27      |Savings   |New York|Failed            |
|5  |2023-01-05 00:00:00|CUST005    |15109.98          |Purchase        |Bank Transfer |285987.99      |Electronics|Houston |Pending           |
+---+-------------------+----------+------------------+----------------+--------------+--------------+----------+--------+------------------+
```

Driver logs


Untitled Notebook 2026-02-19
Logs - Databricks
adb-7405616304681582.2.azuredatabricks.net/editor/notebooks/2173315967171657?o=7405616304681582#command/4838369930516258
Microsoft Azure
databricks
Search data, notebooks, recents, and more...
CTRL + P
New
Workspace
Recents
Catalog
Jobs & Pipelines
Compute
Marketplace
SQL
SQL Editor
Queries
Dashboards
Genie
Alerts
Query History
SQL Warehouses
Data Engineering
Runs
Data Ingestion
Catalog
Type to search...
For you
All
My organization
databricks_ws_c53f8853_c48b_4e42...
system
Delta Shares Received
samples
Legacy
hive_metastore
Untitled Notebook 2026-02-19 11:40:11
File
Edit
View
Run
Help
Python
Tabs: ON
Last edit was 1 minute ago
Run all
Serverless
Just now (2s)
1
Python
df = spark.read.table("databricks_ws_c53f8853_c48b_4e42_8e0b_b49d1134d1bd.default.transactions")
df: pyspark.sql.connect.dataframe.DataFrame = [ID: integer, Date: timestamp ... 8 more fields]
Just now (2s)
2
df.show(5, truncate=False)
See performance (1)
+---+-------------------+-----------+------------------+---------------+--------------+---------------+-----------+--------+------------------+
|ID |Date |Customer_ID|Transaction_Amount|Transaction_Type|Payment_Method|Account_Balance|Category |Location|Transaction_Status|
+---+-------------------+-----------+------------------+---------------+--------------+---------------+-----------+--------+------------------+
1	2023-01-01 00:00:00	CUST001	27337.49	Transfer	Bank Transfer	804821.06	Electronics	New York	Pending
2	2023-01-02 00:00:00	CUST002	97716.6	Purchase	Credit Card	540169.76	Electronics	Houston	Pending
3	2023-01-03 00:00:00	CUST003	5752.36	Transfer	Debit Card	925251.6	Electronics	New York	Completed
4	2023-01-04 00:00:00	CUST004	93443.22	Transfer	Debit Card	704136.27	Savings	New York	Failed
5	2023-01-05 00:00:00	CUST005	15109.98	Purchase	Bank Transfer	285987.99	Electronics	Houston	Pending
+---+-------------------+-----------+------------------+---------------+--------------+---------------+-----------+--------+------------------+
only showing top 5 rows
[Shift+Enter] to run and move to next cell

Let's practice!

DATA TRANSFORMATION WITH SPARK SQL IN DATABRICKS

Shaping data with PySpark and SQL

DATA TRANSFORMATION WITH SPARK SQL IN DATABRICKS



Disha Mukherjee
Lead Data Engineer

datacamp

Shaping your data

```
path = "/Volumes/.../default/transactions.csv"
df = spark.read.csv(path, header=True, inferSchema=True)
df.show(5, truncate=False)
```

```
+---+-------------------+----------+------------------+---------------+--------------+---------------+----------+--------+------------------+
|ID |Date               |Customer_ID|Transaction_Amount|Transaction_Type|Payment_Method|Account_Balance|Category  |Location|Transaction_Status|
+---+-------------------+----------+------------------+---------------+--------------+---------------+----------+--------+------------------+
|1  |2023-01-01 00:00:00|CUST001    |27337.49          |Transfer       |Bank Transfer |804821.06      |Electronics|New York|Pending           |
|2  |2023-01-02 00:00:00|CUST002    |97716.6           |Purchase       |Credit Card   |540169.76      |Electronics|Houston |Pending           |
|3  |2023-01-03 00:00:00|CUST003    |5752.36           |Transfer       |Debit Card    |925251.6       |Electronics|New York|Completed         |
|4  |2023-01-04 00:00:00|CUST004    |93443.22          |Transfer       |Debit Card    |704136.27      |Savings   |New York|Failed            |
|5  |2023-01-05 00:00:00|CUST005    |15109.98          |Purchase       |Bank Transfer |285987.99      |Electronics|Houston |Pending           |
+---+-------------------+----------+------------------+---------------+--------------+---------------+----------+--------+------------------+
```

- `select()` → choose columns
- `filter()` → choose rows

Selecting columns

```
df.select("ID", "Customer_ID", "Transaction_Amount", "Category").show(5)
```

```
+---+-----------+------------------+-----------+
| ID|Customer_ID|Transaction_Amount|   Category|
+---+-----------+------------------+-----------+
|  1|    CUST001|          27337.49|Electronics|
|  2|    CUST002|           97716.6|Electronics|
|  3|    CUST003|           5752.36|Electronics|
|  4|    CUST004|          93443.22|    Savings|
|  5|    CUST005|          15109.98|Electronics|
+---+-----------+------------------+-----------+
```

Filtering rows

```
from pyspark.sql import functions as F

filtered_df = df.filter(F.col("Transaction_Status") == "Completed")
filtered_df.show(5)
```

```
+---+-------------------+-----------+------------------+------------------+
| ID|               Date|Customer_ID|Transaction_Amount|Transaction_Status|
+---+-------------------+-----------+------------------+------------------+
|  3|2023-01-03 00:00:00|    CUST003|           5752.36|         Completed|
|  9|2023-01-09 00:00:00|    CUST009|          28959.12|         Completed|
| 10|2023-01-10 00:00:00|    CUST010|          72098.18|         Completed|
| 11|2023-01-11 00:00:00|    CUST011|          49771.05|         Completed|
| 20|2023-01-20 00:00:00|    CUST020|          69825.14|         Completed|
+---+-------------------+-----------+------------------+------------------+
```

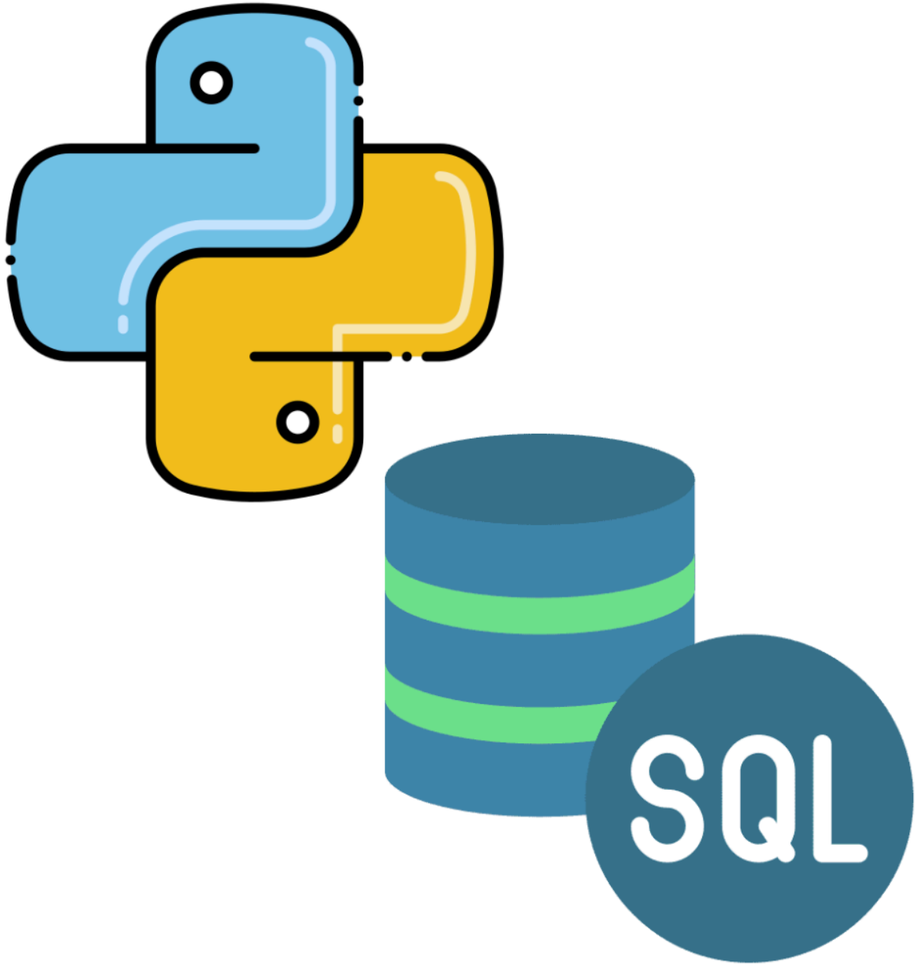
Combining filter conditions

```
high_value_df = df.filter(
    (F.col("Transaction_Amount") > 20000) &
    (F.col("Transaction_Status") == "Completed")
)
high_value_df.show(3)
```

```
+---+-------------------+----------+------------------+------------------+
| ID|               Date|Customer_ID|Transaction_Amount|Transaction_Status|
+---+-------------------+----------+------------------+------------------+
|  9|2023-01-09 00:00:00|    CUST009|          28959.12|         Completed|
| 10|2023-01-10 00:00:00|    CUST010|          72098.18|         Completed|
| 11|2023-01-11 00:00:00|    CUST011|          49771.05|         Completed|
+---+-------------------+----------+------------------+------------------+
```

Python methods or SQL

- **Python methods:** `select()` , `filter()`
  - Build transformations step by step



- **SQL queries:** Standard SQL syntax
  - More readable for aggregations and grouping

Creating a temporary view

```
df.createOrReplaceTempView("transactions")
```

- **Temporary view** = name that points to DataFrame
- Lasts only for the current Spark session

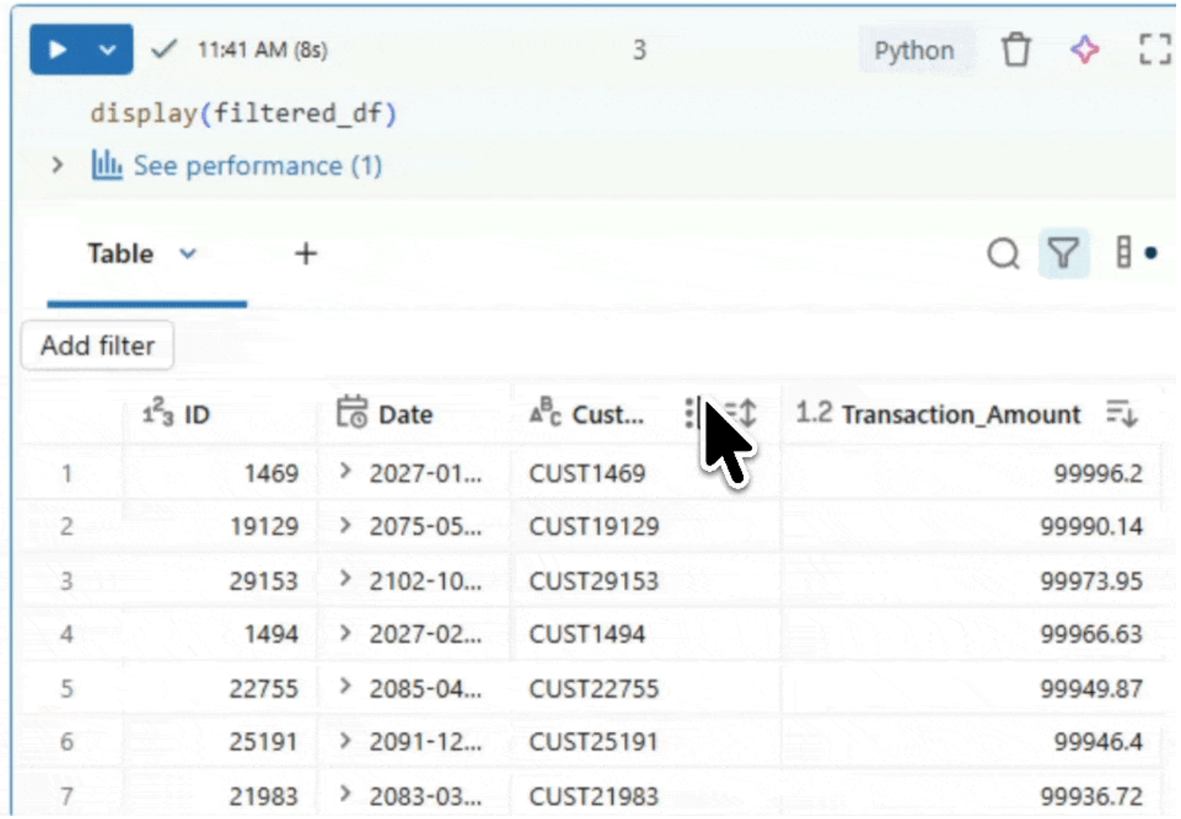
Querying with SQL

```
%sql
SELECT Category, COUNT(*) AS transaction_count
FROM transactions
WHERE Transaction_Status = 'Completed'
GROUP BY Category
ORDER BY transaction_count DESC
```

```
+-----------+-----------------+
|   Category|transaction_count|
+-----------+-----------------+
|   Clothing|             6764|
|     Dining|             6732|
|Electronics|             6640|
|    Savings|             6629|
|  Groceries|             6557|
+-----------+-----------------+
```

Validating with display()

```
display(filtered_df)
```



- Sort, page through results
- Spot nulls and outliers

Inspecting query plans

```
filtered_df.explain()
```

```
== Physical Plan ==
*(1) Filter (Transaction_Status = Completed)
+- FileScan csv [ID, Date, Customer_ID, ...]
```

- Also works with SQL: `EXPLAIN SELECT ...`
- Useful for fixing performance issues

Reading Spark errors

```
df.select("Account").show()
```

```
[UNRESOLVED_COLUMN.WITH_SUGGESTION] A column with name `Account`
cannot be resolved. Did you mean one of the following?
[`Date`, `Location`, `Customer_ID`, `Transaction_Amount`]
```

Let's practice!

DATA TRANSFORMATION WITH SPARK SQL IN DATABRICKS